

Rapport de projet informatique

KyotoBot - Assistant touristique intelligent pour Kyoto

Projet réalisé du 22 novembre 2024 au 6 janvier 2025

Membres du groupe

Yasmine Boulil - N° 45004303

Kholef Achour - N° 45006563

Remerciements

Nous tenons à remercier notre enseignant pour ses conseils techniques et méthodologiques prodigués.

Nous remercions également Anthropic et Mistral AI pour la mise à disposition de leurs API d'intelligence artificielle, qui ont été essentielles à la réalisation de ce projet.

Lien du dépôt GitHub : <https://github.com/yasmineboulil/kyotobot-projet.git>

Table des matières

1	Introduction	5
2	Environnement de travail	5
2.1	Matériel et logiciels	5
2.2	Outils de développement	5
2.3	Méthodologie de travail	5
3	Description du projet et objectifs	6
3.1	Problématique	6
3.2	Objectifs du projet	6
3.3	Public cible	6
4	Bibliothèques, Outils et technologies	6
4.1	Architecture générale	6
4.2	Backend : Flask	7
4.3	Frontend : HTML5, CSS3, JavaScript	7
4.3.1	HTML5	7
4.3.2	CSS3	7
4.3.3	JavaScript	8
4.4	Intelligence Artificielle : Mistral AI	8
4.4.1	Choix du modèle	8
4.4.2	Architecture de l'IA conversationnelle	8
4.5	Utilisation d'un assistant IA dans le développement	9
4.5.1	Contexte et méthodologie	9
4.6	Base de données	10
5	Travail réalisé	11
5.1	Fonctionnalités réalisées	11
5.2	Fonctionnalités non réalisées	12
5.3	Répartition du travail	12
6	Difficultés rencontrées	12
6.1	Intégration de l'API Mistral AI	12
6.2	Extraction automatique d'informations	12
6.3	Gestion des z-index et superposition d'éléments	13
6.4	Performance et temps de réponse du chatbot	13
6.5	Débogage du JavaScript asynchrone	13
7	Bilan	13
7.1	Conclusion	13
7.2	Perspectives d'évolution	14
8	Bibliographie	16
9	Webographie	17
10	Annexes	18

A	Cahier des charges	18
A.1	Objectif général	18
A.2	Contraintes techniques	18
A.3	Fonctionnalités attendues	18
B	Exemple d'exécution du projet	18
B.1	Page d'accueil	18
B.2	Chatbot - Message de bienvenue	19
B.3	Exemple de conversation simple	20
B.4	Exemple d'itinéraire complet	20
B.5	Guide des lieux	21
B.6	Filtres par catégorie	21
B.7	Page de détail d'un lieu	21
B.8	Itinéraires prédéfinis	23
B.9	Détail d'un itinéraire	23
B.10	Connexion Chat Lieux	24
C	Manuel utilisateur	24
C.1	Installation et démarrage	24
C.2	Utilisation du chatbot	25
C.3	Navigation dans le guide des lieux	25
C.4	Consultation des itinéraires prédéfinis	26
C.5	Navigation générale	26
C.6	Conseils d'utilisation	26

1 Introduction

Le tourisme à Kyoto, ancienne capitale impériale du Japon, attire chaque année des millions de visiteurs désireux de découvrir ses temples millénaires, ses jardins zen et sa culture traditionnelle. Face à l'abondance de sites touristiques (plus de 2000 temples et sanctuaires), les visiteurs se trouvent souvent démunis pour organiser efficacement leur séjour en fonction de leurs contraintes de temps et de budget.

Ce projet vise à développer un assistant touristique intelligent, KyotoBot, capable de générer des itinéraires personnalisés grâce à l'intelligence artificielle. L'application combine une interface web intuitive, une base de données structurée de lieux emblématiques, et un chatbot conversationnel alimenté par l'API Mistral AI.

L'objectif principal est de permettre aux utilisateurs de planifier leur visite de Kyoto de manière optimale, en tenant compte de leurs préférences (temples, gastronomie, nature), de leur budget et de la durée de leur séjour. Le système extrait automatiquement ces informations des messages utilisateurs et propose des recommandations adaptées.

Ce rapport présente l'architecture technique du projet, les choix technologiques effectués, le travail réalisé, ainsi qu'une réflexion sur l'utilisation de l'intelligence artificielle tant dans le développement que dans le fonctionnement de l'application.

2 Environnement de travail

2.1 Matériel et logiciels

Le développement du projet a été réalisé dans l'environnement suivant :

- **Système d'exploitation** : Windows 11
- **Éditeur de code** : Visual Studio Code avec extensions Python et HTML/CSS
- **Navigateurs de test** : Google Chrome, Mozilla Firefox
- **Gestionnaire de versions** : Git et GitHub
- **Environnement Python** : Python 3.10+
- **Gestionnaire de paquets** : pip

2.2 Outils de développement

- **Framework web** : Flask (Python) pour le backend
- **API d'intelligence artificielle** : Mistral AI (mistral-small-latest)
- **Assistant de développement** : Claude (Anthropic) pour l'aide au code et l'architecture
- **Documentation** : README.md (Markdown)
- **Design** : CSS3 avec animations et transitions

2.3 Méthodologie de travail

Le projet a été développé de manière itérative :

1. Conception de l'architecture et choix des technologies
2. Développement du chatbot et intégration de l'API Mistral
3. Création de la base de données de lieux et itinéraires
4. Développement de l'interface utilisateur
5. Tests et amélioration du design

3 Description du projet et objectifs

3.1 Problématique

Kyoto compte plus de 2000 temples et sanctuaires, ainsi qu’une multitude de jardins, restaurants et quartiers traditionnels. Les touristes, souvent limités en temps (1 à 3 jours en moyenne) et en budget, ont du mal à :

- Identifier les lieux correspondant à leurs centres d’intérêt
- Optimiser leurs déplacements pour minimiser les temps de transport
- Estimer les coûts totaux de leur visite
- Obtenir des conseils pratiques adaptés à leurs contraintes

Les guides touristiques traditionnels offrent des informations exhaustives mais peu personnalisées. Les applications existantes manquent souvent d’intelligence conversationnelle pour comprendre les demandes en langage naturel.

3.2 Objectifs du projet

Notre projet vise à créer un assistant touristique intelligent qui :

1. **Comprend les demandes en langage naturel** : L’utilisateur peut poser des questions comme ”J’ai 50€ et 4 heures, je veux voir des temples”, et le système extrait automatiquement les informations pertinentes.
2. **Génère des itinéraires personnalisés** : En fonction du budget, de la durée et des centres d’intérêt détectés, l’application propose un parcours optimisé avec horaires, durées de visite et coûts.
3. **Base de données riche et structurée** : 35 lieux emblématiques de Kyoto sont répertoriés avec descriptions détaillées, prix d’entrée, conseils pratiques et liens vers Google Maps. Les lieux sont catégorisés (temples, jardins, restaurants, quartiers, marchés, hébergements).
4. **Réponses adaptatives** : Le chatbot adapte le format de ses réponses selon le type de demande : réponse courte pour une question simple, itinéraire détaillé pour une demande de planification.
5. **Navigation fluide** : Connexions intelligentes entre les différentes sections (possibilité d’interroger le chatbot depuis une page de lieu, pré-remplissage des questions).

3.3 Public cible

L’application s’adresse principalement aux touristes francophones visitant Kyoto pour la première fois, avec des contraintes de temps et de budget variables.

4 Bibliothèques, Outils et technologies

4.1 Architecture générale

Le projet suit une architecture MVC (Modèle-Vue-Contrôleur) adaptée à Flask :

- **Modèle** : Base de données structurée en Python (data.py) contenant lieux et itinéraires
- **Vue** : Templates HTML avec Jinja2 pour le rendu dynamique
- **Contrôleur** : Routes Flask organisées en blueprints (home, chat, lieux, itinéraires)
- **Services** : Logique métier encapsulée (ai_service.py pour l'interaction avec Mistral AI)

4.2 Backend : Flask

Flask est un micro-framework web Python choisi pour sa simplicité et sa flexibilité. Il permet de créer rapidement des applications web sans imposer de structure rigide.

Fonctionnalités utilisées :

- **Blueprints** : Organisation modulaire du code permettant de séparer les routes par fonctionnalité (home, chat, lieux, itinéraires). Chaque blueprint est un module indépendant facilitant la maintenance.
- **Routes** : Gestion des URLs et des requêtes HTTP (GET pour l'affichage des pages, POST pour les interactions du chatbot).
- **Templates Jinja2** : Système de templating permettant d'injecter dynamiquement des données Python dans les pages HTML (boucles for pour afficher les lieux, conditions if pour les filtres).
- **Fichiers statiques** : Service automatique des fichiers CSS, JavaScript et images via le dossier static/.

Avantages de Flask :

- Courbe d'apprentissage rapide, idéal pour un projet pédagogique
- Grande communauté et documentation abondante en ligne
- Léger et performant pour une application de taille moyenne
- Extensible via des extensions (Flask-SQLAlchemy, Flask-Login si évolution future)

4.3 Frontend : HTML5, CSS3, JavaScript

4.3.1 HTML5

Structure sémantique des pages avec balises modernes (header, nav, section, article) améliorant l'accessibilité et le référencement.

4.3.2 CSS3

Design moderne intégrant :

- **Animations CSS** : @keyframes pour les cerisiers tombants (effet visuel japonais), transitions fluides sur les cartes et boutons
- **Flexbox et Grid** : Positionnement responsive des éléments (grille de lieux, cartes d'itinéraires, layout du chatbot)
- **Dégradés linéaires** : Navbar avec dégradé noir-rouge-noir, boutons avec dégradés colorés
- **Media queries** : Adaptation mobile et tablette (breakpoints à 768px et 1024px)

- **Effets hover** : Transformations au survol (scale, translateY) pour améliorer l'interactivité
- **Palette de couleurs** : Rouge japonais traditionnel (#8B0000, #c62828), noir (#1a1a1a) et or (#D4AF37) pour un rendu mature et élégant

4.3.3 JavaScript

- **Gestion des événements** : Écoute des clics, soumission de formulaires, interactions utilisateur
- **Fetch API** : Requêtes HTTP asynchrones vers le backend Flask pour envoyer les messages du chat et recevoir les réponses de l'IA sans recharger la page
- **Manipulation du DOM** : Ajout dynamique des messages dans la conversation, affichage de l'indicateur "en train d'écrire"
- **Marked.js** : Bibliothèque pour convertir le Markdown des réponses de l'IA en HTML formaté (listes, gras, italique, liens)
- **Paramètres URL** : Récupération du paramètre ?q= pour pré-remplir le chat depuis d'autres pages

4.4 Intelligence Artificielle : Mistral AI

4.4.1 Choix du modèle

Notre chatbot repose sur l'API Mistral AI, spécifiquement le modèle `mistral-small-latest`. Ce choix s'explique par :

- **Performance/coût** : Équilibre optimal pour un projet étudiant (coût raisonnable, performance suffisante)
- **Compréhension du français** : Excellente maîtrise de la langue française, cruciale pour notre public cible
- **Latence acceptable** : Réponses rapides (2-5 secondes) pour une expérience chatbot fluide
- **Suivre des instructions** : Capacité à respecter des prompts système complexes (prompt engineering)

4.4.2 Architecture de l'IA conversationnelle

1. Prompt engineering

Nous avons conçu un prompt système de 50 lignes définissant précisément :

- Le rôle de KyotoBot (guide touristique expert de Kyoto uniquement)
- Les règles de réponse (analyser budget/durée/intérêts, proposer itinéraires optimisés)
- La détection du type de demande (question simple vs demande de planification)
- Les formats de réponse attendus selon le type de demande
- La liste des 35 lieux disponibles dans la base de données

2. Extraction automatique d'informations

Avant d'envoyer le message à l'API, notre système analyse le texte utilisateur avec des expressions régulières (regex) pour extraire :

- **Budget** : Patterns `(\d+)\s*€, (\d+)\s*euros?, (\d+)\s*¥`
- **Durée** : Patterns `(\d+)\s*jours?, (\d+)\s*heures?, (\d+)\s*h`

- **Centres d'intérêt** : Détection de mots-clés (temple, jardin, restaurant, nature, culture, shopping)
- **Période de visite** : Détection de saisons (printemps/cerisiers, automne/momiji, été, hiver)

Cette extraction permet d'enrichir le contexte envoyé à l'IA sans que l'utilisateur ait à remplir un formulaire structuré.

3. Enrichissement du contexte

Le message utilisateur est enrichi avec :

- Les informations extraites formatées en [CONTEXTE DÉTECTÉ]
- Les lieux pertinents de la base de données selon les intérêts détectés (filtrage par catégorie)
- L'historique complet de la conversation pour maintenir la cohérence des échanges

4. Gestion de la mémoire conversationnelle

L'historique des messages (utilisateur et assistant) est maintenu en mémoire durant toute la session. Cela permet au chatbot de faire référence aux échanges précédents et d'affiner progressivement ses recommandations.

5. Recommandations hybrides

L'approche combine :

- **Connaissances générales** : L'IA Mistral possède des connaissances générales sur Kyoto (culture, histoire, géographie)
- **Base de données structurée** : Nos 35 lieux avec prix, horaires et conseils précis garantissent des informations factuellement correctes

Cette hybridation évite les hallucinations de l'IA (invention de lieux inexistants) tout en bénéficiant de sa capacité de génération de langage naturel.

Exemple de traitement complet :

Message utilisateur : *"J'ai 60€ et une journée, j'adore les temples"*

- Extraction** : Budget=60€, Durée=1 jour, Intérêt=temples
- Filtrage base de données** : Sélection des 7 temples disponibles
- Enrichissement** : Ajout du contexte + liste des temples (Kiyomizu-dera, Fushimi Inari, Kinkaku-ji, etc.)
- Envoi à Mistral AI** : Prompt système + historique + message enrichi
- Réception** : Itinéraire personnalisé avec horaires précis, temps de transport, budget détaillé (entrées + repas), conseils pratiques

4.5 Utilisation d'un assistant IA dans le développement

4.5.1 Contexte et méthodologie

Dans le cadre de ce projet, nous avons exploré l'utilisation de Claude (Anthropic) comme outil d'aide au développement, conformément aux pratiques modernes de l'industrie logicielle où les développeurs s'appuient sur des assistants IA pour optimiser leur productivité.

Méthodologie adoptée :

- Phase de conception** : Nous avons défini nous-mêmes l'architecture globale du projet (MVC avec Flask, base de données structurée, intégration API

Mistral AI, organisation en blueprints). Les décisions d'architecture ont été prises après réflexion sur nos contraintes et objectifs.

- (b) **Validation technique** : Claude a servi de référence technique pour valider nos choix et suggérer des bonnes pratiques reconnues (séparation des responsabilités, gestion d'erreurs, organisation du code).
- (c) **Accélération du développement** : Pour certaines tâches répétitives ou techniques (écriture de regex, structure CSS complexe, formatage de données), l'IA a permis de gagner du temps sur l'implémentation.
- (d) **Résolution de problèmes** : Lorsque nous rencontrons des bugs (conflits z-index, problèmes responsive, gestion des erreurs API), Claude aidait à identifier les causes et proposer des pistes de solution que nous testions et adaptons.

Approche critique maintenue :

- Chaque portion de code générée avec l'aide de l'intelligence artificielle a été relue, comprise et testée avant d'être intégrée au projet final.
- Les suggestions inadaptées à nos besoins ont été modifiées ou rejetées
- Nous avons effectué nos propres recherches complémentaires pour comprendre les concepts (documentation Flask, tutoriels CSS Grid, spécifications API Mistral)

Exemples concrets d'utilisation :

- **Architecture** : Consultation sur l'organisation des blueprints Flask et la séparation des responsabilités
- **Conception du prompt IA** : amélioration progressive du prompt système afin de guider le modèle vers des réponses structurées et cohérentes, en lien avec les données disponibles dans la base.
- **Extraction d'informations** : Aide à la conception des patterns regex pour détecter budget et durée dans les messages
- **Design CSS** : Suggestions sur la palette de couleurs (rouge japonais + noir + or) et les animations
- **Débogage** : Identification d'un problème de z-index empêchant les clics sur les boutons du chat

Bilan :

L'utilisation d'un assistant IA s'est révélée bénéfique pour accélérer le développement tout en maintenant une démarche pédagogique active. Cela reflète les pratiques actuelles de l'industrie où les développeurs utilisent ces outils comme compléments à leurs compétences. Cette expérience nous a permis de développer notre capacité à évaluer critiqueusement les suggestions automatiques et à les adapter à nos besoins spécifiques.

4.6 Base de données

La base de données est implémentée sous forme de structures Python dans le fichier `data.py` :

- **LIEUX** : Liste de 35 dictionnaires Python, chacun contenant :
 - id (identifiant unique)
 - nom (nom du lieu)
 - categorie (temple, jardin, marche, quartier, restaurant, hébergement)

- description (texte détaillé)
 - duree (temps de visite estimé)
 - prix (coût d'entrée ou fourchette de prix)
 - conseil (recommandations pratiques)
 - quartier (localisation géographique)
 - maps (lien Google Maps)
 - **ITINERAIRES** : Liste de 5 dictionnaires d'itinéraires prédéfinis, chacun contenant :
 - id, nom, emoji, duree, budget
 - description (présentation de l'itinéraire)
 - etapes (liste détaillée des lieux avec horaires et notes)
 - conseils (recommandations spécifiques)
 - **CATEGORIES** : Liste des 6 catégories avec id, nom et emoji associé
- Ce format Python facilite la manipulation des données (filtrage, recherche) tout en restant lisible et maintenable. Pour une évolution future, la migration vers une base de données relationnelle (SQLite, PostgreSQL) serait envisageable.

5 Travail réalisé

5.1 Fonctionnalités réalisées

Toutes les fonctionnalités prévues ont été implémentées avec succès :

1. Page d'accueil enrichie

- Interface animée avec cerisiers tombants et dégradé zen
- Section explicative présentant les 3 fonctionnalités principales (Chat, Lieux, Itinéraires)
- Cartes visuelles avec icônes et exemples concrets
- Trois boutons d'action clairement identifiés
- Navbar fixe pour navigation rapide

2. Chatbot intelligent (KyotoBot)

- Conversation en langage naturel sans formulaire contraint
- Extraction automatique de budget, durée, intérêts et période de visite
- Réponses adaptatives : courtes pour questions simples, détaillées pour demandes de planification
- Mémoire conversationnelle maintenue durant toute la session
- Gestion d'erreurs : refus poli si demande hors-sujet Kyoto
- Interface moderne avec indicateur "en train d'écrire"
- Boutons de suggestions rapides pour faciliter la prise en main

3. Guide des lieux

- Base de données de 35 lieux emblématiques répartis en 6 catégories
- Système de filtres par catégorie (temples, jardins, marchés, quartiers, restaurants, hébergements)
- Affichage en grille responsive avec cartes visuelles
- Page de détail pour chaque lieu avec informations complètes
- Liens Google Maps pour chaque lieu
- Bouton "Demander plus d'infos à KyotoBot" pré-remplissant le chat avec une question sur le lieu

4. Itinéraires prédéfinis

- 5 itinéraires thématiques complets (Kyoto en 1 jour, Circuit temples, Kyoto gourmand, Nature & Zen, Kyoto Instagram)
- Chaque itinéraire avec présentation, durée totale, budget estimé
- Détails pour chaque étape : horaire, lieu, durée, prix, notes spécifiques
- Conseils pratiques globaux pour chaque itinéraire
- Possibilité de personnaliser l'itinéraire via le chatbot

5. Design et navigation

- Navbar fixe avec navigation fluide entre toutes les pages
- Palette de couleurs mature : rouge japonais, noir, or
- Animations CSS : cerisiers tombants, transitions fluides, effets hover sophistiqués
- Responsive design : adaptation mobile et tablette avec media queries
- Connexions intelligentes entre pages (lieux → chat, itinéraires → chat)

5.2 Fonctionnalités non réalisées

- **Système de favoris** : Permettre de sauvegarder des lieux préférés (non prioritaire pour un MVP)
- **Export PDF d'itinéraire** : Télécharger un itinéraire en PDF (complexité technique vs temps disponible)
- **Mode multilingue** : Anglais et japonais (focus sur le public francophone)

5.3 Répartition du travail

Yasmine Boulil : Développement du backend (Flask, routes, services), intégration de l'API Mistral AI, design et animations, documentation (rapport).

Kholef Achour : Développement du frontend (HTML/CSS/JS), documentation (README), tests et débogage.

6 Difficultés rencontrées

6.1 Intégration de l'API Mistral AI

Problème : Compréhension initiale du format des requêtes API et de la gestion du contexte conversationnel. La documentation était claire sur les appels basiques mais moins sur la gestion d'un historique conversationnel.

Solution : Étude approfondie de la documentation officielle Mistral AI, consultation d'exemples sur GitHub, tests itératifs avec différents formats de prompts. Implémentation d'un système de gestion d'historique stockant tous les échanges (utilisateur + assistant) et les réinjectant à chaque nouvelle requête.

Apprentissage : Importance de bien structurer le prompt système et de maintenir un historique cohérent pour des conversations naturelles.

6.2 Extraction automatique d'informations

Problème : Les expressions régulières initiales ne capturaient pas toutes les variations de formulation. Par exemple, "trois heures" n'était pas détecté (seulement "3h" ou "3 heures").

Solution : Création de multiples patterns regex pour couvrir les différentes formulations possibles. Ajout de patterns pour les nombres en toutes lettres aurait nécessité un dictionnaire complet, nous avons donc privilégié les formulations numériques les plus courantes.

Apprentissage : Les regex sont puissantes mais limitées pour le traitement du langage naturel.

6.3 Gestion des z-index et superposition d'éléments

Problème : Les cerisiers animés (décoration) bloquaient les clics sur les boutons du chatbot. L'utilisateur ne pouvait pas interagir avec l'interface. **Solution :** Ajustement des propriétés CSS : `z-index` pour contrôler l'ordre d'empilement, `position: relative/absolute` pour le positionnement, et surtout `pointer-events: none` sur les éléments purement décoratifs pour qu'ils n'interceptent pas les événements souris. **Apprentissage :** Importance de bien comprendre le contexte d'empilement CSS et l'interaction entre position, z-index et pointer-events.

6.4 Performance et temps de réponse du chatbot

Problème : Temps de réponse parfois long de l'API Mistral (3-7 secondes), donnant l'impression que l'application était figée. **Solution :** Ajout d'un indicateur visuel "en train d'écrire..." avec animation de points pulsants. Désactivation temporaire du champ de saisie pendant le traitement. Optimisation des prompts pour réduire la longueur des réponses générées (moins de tokens = temps de génération réduit). **Apprentissage :** L'expérience utilisateur nécessite un feedback visuel constant. Même si le traitement est long, l'utilisateur doit savoir que quelque chose se passe.

6.5 Débogage du JavaScript asynchrone

Problème : Gestion des erreurs lors des appels Fetch API. Si l'API Mistral était indisponible ou renvoyait une erreur, l'application ne réagissait pas clairement. **Solution :** Implémentation de blocs try-catch autour des appels asynchrones. Ajout de messages d'erreur utilisateur conviviaux ("Désolé, une erreur s'est produite. Veuillez réessayer."). Log des erreurs dans la console pour faciliter le débogage. **Apprentissage :** La gestion d'erreurs robuste est cruciale pour les applications web utilisant des API externes. Ne jamais supposer que tout fonctionnera parfaitement.

7 Bilan

7.1 Conclusion

Ce projet nous a permis de développer une application web complète intégrant une intelligence artificielle conversationnelle. Au-delà de l'aspect technique, nous avons acquis des compétences variées touchant plusieurs domaines de l'informatique. **Compétences techniques acquises :**

- **Développement backend** : Maîtrise de Flask, compréhension de l'architecture MVC, gestion des routes et blueprints
- **Intégration d'API externes** : Communication avec l'API Mistral AI, gestion des requêtes HTTP, traitement des réponses JSON
- **Traitement du langage naturel** : Prompt engineering, extraction d'informations avec regex, gestion du contexte conversationnel
- **Développement frontend** : HTML5 sémantique, CSS3 avancé (animations, Grid, Flexbox), JavaScript asynchrone (Fetch API, manipulation DOM)
- **Design d'interface** : Conception d'une expérience utilisateur cohérente, responsive design, animations fluides
- **Gestion de projet** : Organisation du code, versioning Git, documentation

Points forts du projet :

- **Innovation** : L'approche hybride (IA générative + base de données structurée) garantit des recommandations à la fois créatives et factuellement correctes
- **Expérience utilisateur** : L'extraction automatique d'informations évite les formulaires fastidieux. L'utilisateur interagit naturellement en langage courant.
- **Complétude** : Le site couvre l'ensemble du parcours touristique (planification via chat, exploration des lieux, découverte d'itinéraires)
- **Design soigné** : Interface moderne et élégante reflétant l'esthétique japonaise

Réflexion sur l'utilisation de l'IA : L'utilisation de Claude comme assistant de développement a été une expérience formatrice. Elle nous a appris à :

- Évaluer critiqueusement les suggestions automatiques
- Adapter les solutions proposées à nos besoins spécifiques
- Maintenir une démarche d'apprentissage active malgré l'aide extérieure

Cette approche reflète les pratiques actuelles de l'industrie où les développeurs s'appuient sur des outils d'IA pour optimiser leur productivité tout en conservant leur expertise et leur capacité de décision.

7.2 Perspectives d'évolution

Plusieurs améliorations pourraient être apportées au projet dans une version future : **Enrichissement de la base de données** :

- Passer de 35 à 100+ lieux avec photos
- Ajouter des hébergements variés (ryokans, hôtels, auberges)
- Intégrer des événements saisonniers (festivals, illuminations)
- Enrichir avec avis et notes d'utilisateurs

Personnalisation avancée :

- Système de comptes utilisateurs avec historique des conversations
- Sauvegarde des préférences et suggestions basées sur l'historique
- Système de favoris avec localStorage ou base de données
- Recommandations collaboratives (popularité des lieux)

Fonctionnalités collaboratives :

- Partage d'itinéraires entre utilisateurs (génération de liens partageables)
- Système d'évaluation et de commentaires sur les lieux
- Forum ou section QA pour la communauté

Optimisation des déplacements :

- Intégration d'une API de calcul d'itinéraires (Google Maps Directions API)

- Optimisation automatique de l'ordre de visite selon la géolocalisation
- Estimation précise des temps de transport (bus, métro, marche)
- Affichage sur carte interactive avec tracé de l'itinéraire

Accessibilité et internationalisation :

- Mode hors-ligne avec cache des informations essentielles (PWA)
- Support multilingue (anglais, japonais) avec modèle IA multilingue
- Mode sombre pour réduire la fatigue oculaire
- Amélioration de l'accessibilité (lecteurs d'écran, navigation clavier)

Fonctionnalités avancées :

- Export d'itinéraire en PDF téléchargeable
- Synchronisation avec Google Calendar pour planification
- Notifications push pour rappels (horaires de visite, réservations)
- Intégration de la réalité augmentée (informations contextuelles sur site)

Aspects techniques :

- Migration vers une base de données relationnelle (PostgreSQL)
- Système de cache pour réduire les appels API et améliorer les performances
- Tests automatisés (unittest, pytest) pour garantir la stabilité
- Déploiement sur serveur cloud (Heroku, AWS, Google Cloud)
- Monitoring et analytics pour comprendre l'usage réel

Ces évolutions transformeraient KyotoBot d'un prototype pédagogique en une application touristique complète et professionnelle, tout en conservant sa simplicité d'utilisation et son intelligence conversationnelle.

8 Bibliographie

- [1] Flask Documentation, *Quickstart Guide et API Reference*, Pallets Projects, <https://flask.palletsprojects.com/>
- [2] Mistral AI Documentation, *API Reference et Prompt Engineering Guide*, Mistral AI, <https://docs.mistral.ai/>
- [3] Mozilla Developer Network, *Web Technologies Documentation (HTML, CSS, JavaScript)*, Mozilla Foundation, <https://developer.mozilla.org/>
- [4] Python Documentation, *The Python Standard Library*, Python Software Foundation, <https://docs.python.org/3/>
- [5] Jinja Documentation, *Template Designer Documentation*, Pallets Projects, <https://jinja.palletsprojects.com/>
- [6] Regular Expressions Documentation, *re — Regular expression operations*, Python Software Foundation, <https://docs.python.org/3/library/re.html>
- [7] L. LAMPORT, *L^AT_EX : A Document Preparation System*, Addison-Wesley, 1994

9 Webographie

- [1] GitHub du projet KyotoBot : <https://github.com/yasmineboulil/kyotobot-projet.git>
- [2] CSS-Tricks, *A Complete Guide to Flexbox et Grid*, <https://css-tricks.com/>
- [3] Marked.js Documentation, *Markdown Parser*, <https://marked.js.org/>
- [4] Office du Tourisme de Kyoto, *Informations touristiques officielles*, <https://kyoto.travel/fr/>
- [5] Stack Overflow, *Questions et réponses techniques sur Flask et JavaScript*, <https://stackoverflow.com/>
- [6] GitHub Guides, *Understanding the GitHub flow*, <https://guides.github.com/>
- [7] W3Schools, *Tutoriels HTML, CSS et JavaScript*, <https://www.w3schools.com/>

10 Annexes

1Annexe :

A Cahier des charges

A.1 Objectif général

Développer une application web d’assistance touristique pour la ville de Kyoto utilisant l’intelligence artificielle pour générer des itinéraires personnalisés en fonction des préférences, du budget et du temps disponible de l’utilisateur.

A.2 Contraintes techniques

- Utilisation obligatoire de Python avec un framework web (Flask retenu)
- Intégration d’une API d’intelligence artificielle externe (Mistral AI retenu)
- Interface web responsive (HTML5, CSS3, JavaScript)
- Versioning avec Git et hébergement sur GitHub
- Documentation complète (README, commentaires code, rapport)

A.3 Fonctionnalités attendues

Fonctionnalités principales :

- (a) Chatbot conversationnel avec compréhension du langage naturel
- (b) Base de données de lieux touristiques avec système de filtres
- (c) Génération d’itinéraires personnalisés selon budget/durée/intérêts
- (d) Itinéraires prédéfinis thématiques pour inspiration

Fonctionnalités secondaires (optionnelles) :

- Liens vers Google Maps pour chaque lieu
- Connexions intelligentes entre les différentes sections
- Animations et design soigné
- Adaptation mobile et tablette

B Exemple d’exécution du projet

Cette annexe présente les principales fonctionnalités de KyotoBot à travers des captures d’écran commentées.

B.1 Page d’accueil

La page d’accueil accueille l’utilisateur avec un design élégant aux couleurs traditionnelles japonaises (rouge, noir, or). Elle présente clairement les trois fonctionnalités principales à travers des cartes visuelles explicatives. // **Éléments visibles :**

- Titre bilingue (japonais et français)
- Section ”Comment ça marche ?” avec 3 cartes explicatives

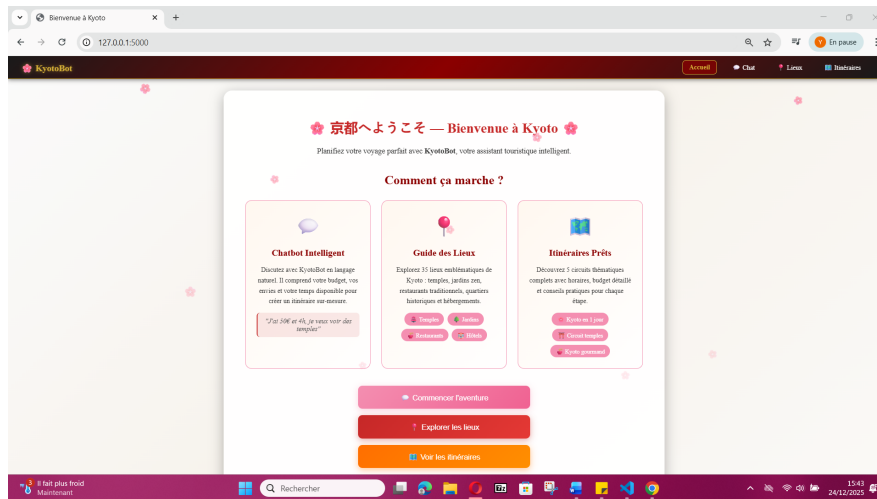


FIGURE 1 – Page d'accueil

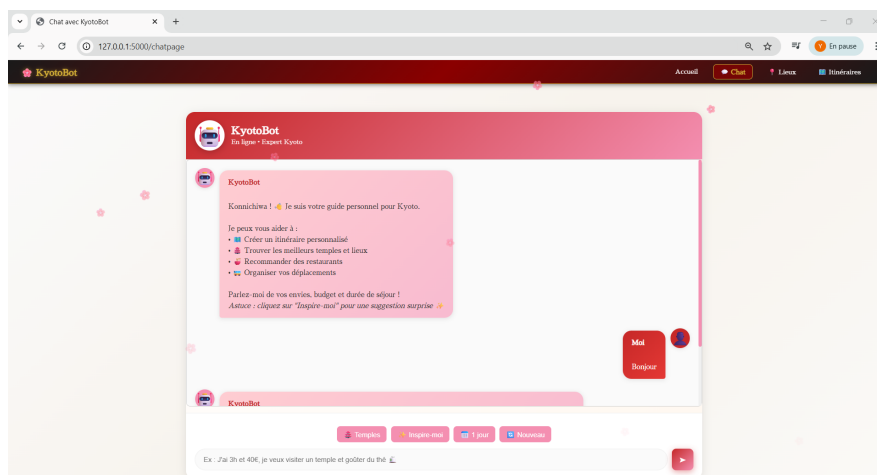


FIGURE 2 – Chatbot

- Trois boutons d'action clairs (Commencer l'aventure, Explorer les lieux, Voir les itinéraires)
- Navbar fixe en haut pour navigation rapide
- Cerisiers animés tombant en arrière-plan

B.2 Chatbot - Message de bienvenue

L'interface du chatbot présente un design moderne de type messagerie. Le bot se présente et explique ses capacités dès l'ouverture. // **Éléments visibles :**

- Zone de messages avec bulles distinctes (bot en rose, utilisateur en rouge)
- Champ de saisie avec placeholder explicatif
- Boutons de suggestions rapides ("Découverte temples", "Inspire-moi", "Itinéraire 1 jour")
- Avatar du bot () et statut "En ligne"

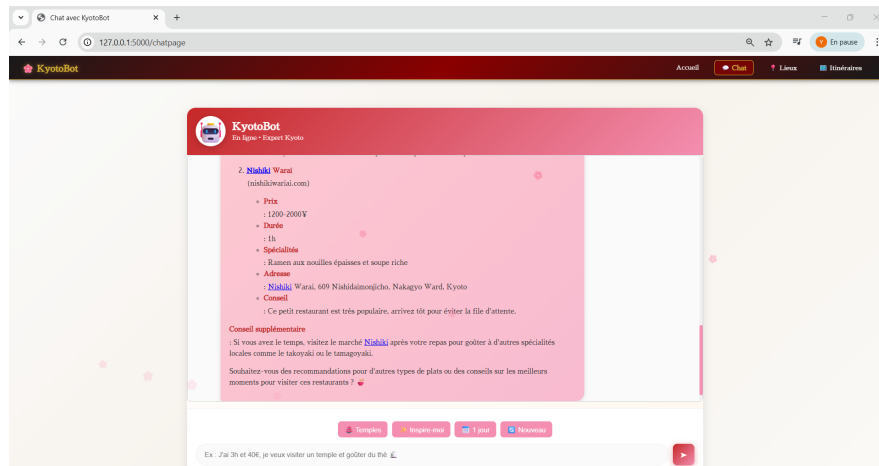


FIGURE 3 – Exemple conversation

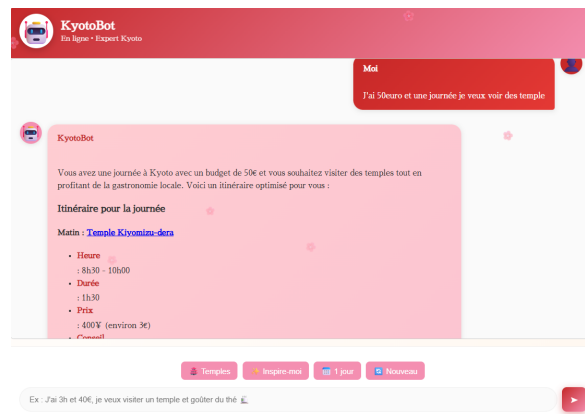


FIGURE 4 – itinéraire complet

B.3 Exemple de conversation simple

Démonstration d'une question simple posée au chatbot : "Où manger des ramens pas chers?". Le bot répond de manière concise avec 3-4 restaurants recommandés. // **Contenu de la réponse :**

- Liste de 3-4 restaurants de ramen avec noms et fourchettes de prix
- Un conseil pratique (horaires d'affluence, plats recommandés)
- Question de suivi : "Voulez-vous que je crée un itinéraire incluant ces lieux?"

B.4 Exemple d'itinéraire complet

L'utilisateur formule une demande de planification : "J'ai 50€ et une journée, je veux voir des temples". Le bot génère un itinéraire détaillé et structuré. //

Contenu de la réponse :

- Résumé de la demande : budget 50€, durée 1 jour, intérêt temples
- Itinéraire chronologique : 8h00 → Temple Fushimi Inari, 10h30 → Temple Kiyomizu-dera, etc.
- Pour chaque étape : horaire, durée de visite, prix d'entrée, notes spécifiques
- Budget détaillé : entrées (1500¥) + repas (1500¥) + transport (700¥) = Total sous budget

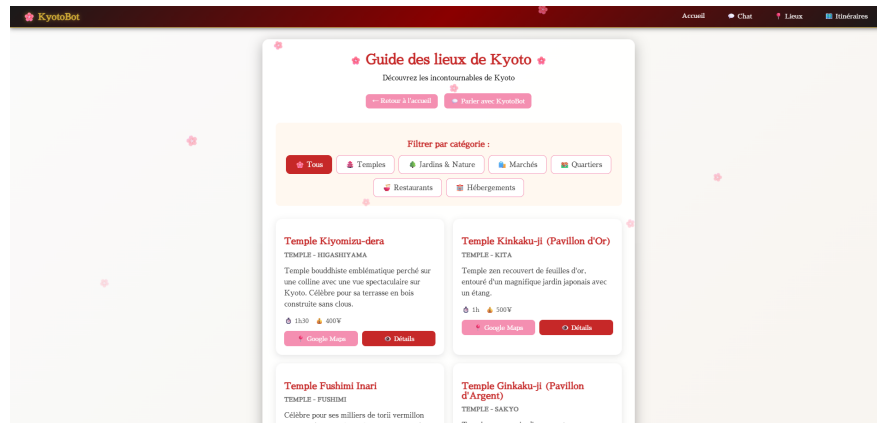


FIGURE 5 – Guide des lieux

- Conseils pratiques : pass bus journalier, chaussures confortables, prévoir de l'eau

B.5 Guide des lieux

Vue d'ensemble de la page "Guide des lieux" montrant la grille de cartes avec les lieux emblématiques de Kyoto. // **Éléments visibles :**

- Barre de filtres par catégorie en haut (Tous, Temples, Jardins, Marchés, Quartiers, Restaurants, Hébergements)
- Grille responsive de cartes de lieux
- Chaque carte affiche : nom, catégorie, description courte, durée, prix, deux boutons (Google Maps, Détails)
- Effet hover : la carte se soulève légèrement au survol

B.6 Filtres par catégorie

Démonstration du système de filtres : l'utilisateur a cliqué sur le filtre "Temples", seuls les 7 temples de la base s'affichent. // **Comportement observé :**

- Le bouton "Temples" est actif (surligné en rouge foncé)
- La grille ne montre que les lieux de catégorie "temple"
- Les autres catégories sont toujours accessibles dans la barre de filtres
- URL mise à jour avec paramètre `?categorie=temple`

B.7 Page de détail d'un lieu

Exemple avec le Temple Kiyomizu-dera. La page affiche toutes les informations détaillées sur ce lieu emblématique. // **Sections visibles :**

- Titre avec nom du lieu
- Section "Informations pratiques" : catégorie, quartier, durée, prix
- Section "Description" : texte détaillé sur l'histoire et l'architecture
- Section "Conseil du guide" : encadré rose avec recommandation pratique
- Deux boutons d'action : "Voir sur Google Maps" et "Demander plus d'infos à KyotoBot"

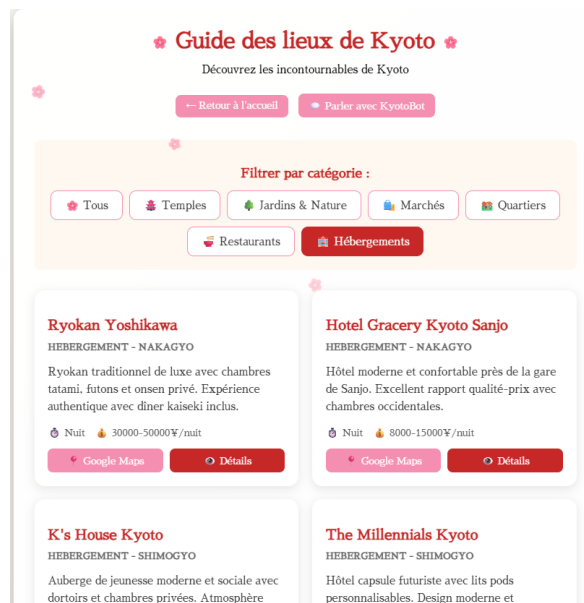


FIGURE 6 – Filtre sur Hébergements

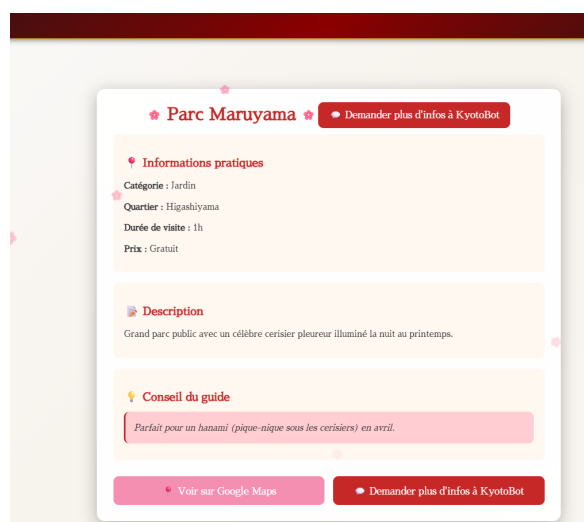


FIGURE 7 – Détails Jardin

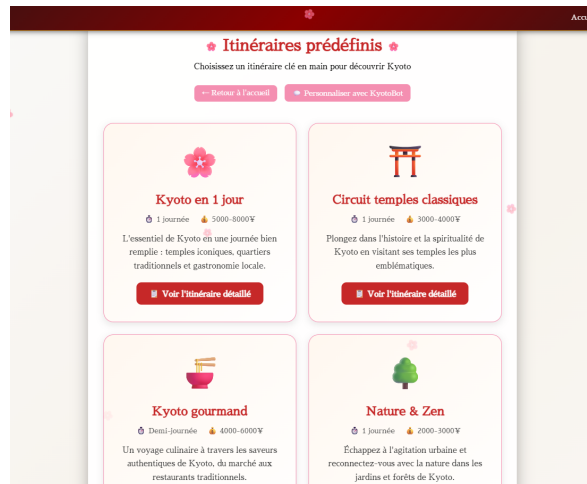


FIGURE 8 – Itinéraire prédéfinis

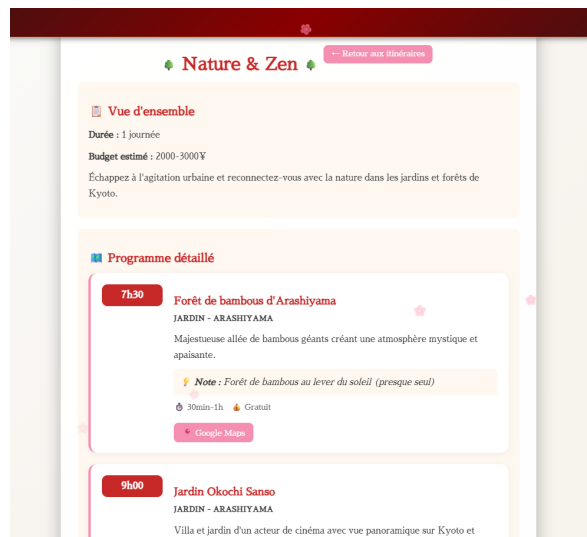


FIGURE 9 – itinéraire détaillé Nature Zen

B.8 Itinéraires prédéfinis

Vue d'ensemble de la page "Itinéraires prédéfinis" présentant les 5 circuits thématiques disponibles. // **Éléments visibles :**

- 5 grandes cartes colorées, une par itinéraire
- Chaque carte affiche : emoji thématique (), nom, durée, budget, description
- Bouton "Voir l'itinéraire détaillé" sur chaque carte
- Design cohérent avec le reste du site

B.9 Détail d'un itinéraire

Exemple avec l'itinéraire "Kyoto en 1 jour". La page présente le programme heure par heure avec toutes les étapes détaillées. // **Contenu affiché :**

- Section "Vue d'ensemble" : durée totale, budget estimé, présentation
- Section "Programme détaillé" : liste chronologique des étapes
- Pour chaque étape : horaire (8h00, 10h30...), nom du lieu, catégorie, description, note spécifique, durée, prix, lien Google Maps

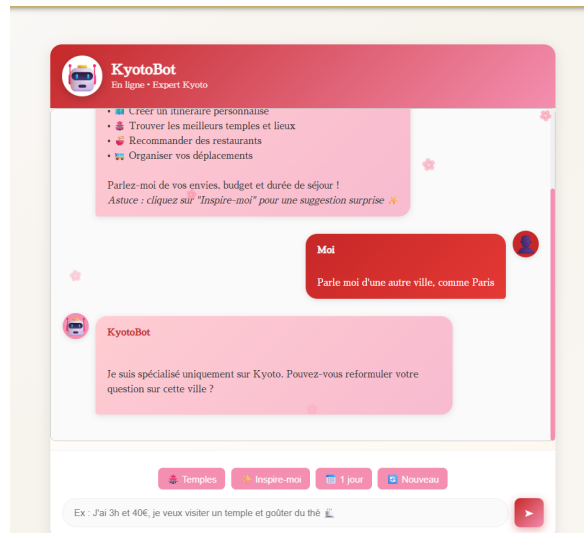


FIGURE 11 – Conversation chatbot spécialisé

- Section "Conseils pratiques" : recommandations globales (pass transport, chaussures, eau)
- Bouton "Personnaliser cet itinéraire avec KyotoBot"

B.10 Connexion Chat Lieux

Démonstration de la connexion intelligente entre les pages. Depuis la page de détail du Temple Kiyomizu-dera, l'utilisateur clique sur "Demander plus d'infos à KyotoBot". // **Comportement observé :**

- Le chatbot s'ouvre avec une question pré-remplie : "Parle-moi de Temple Kiyomizu-dera en détail"
- L'utilisateur peut modifier la question ou l'envoyer directement
- Cette fonctionnalité facilite la transition entre navigation statique et interaction conversationnelle
- Le bot répond avec des informations complémentaires (histoire, anecdotes, conseils pratiques spécifiques)

C Manuel utilisateur

C.1 Installation et démarrage

Prérequis :

- Python 3.10 ou supérieur installé sur votre machine
- Connexion Internet pour les appels API Mistral
- Navigateur web moderne (Chrome, Firefox, Edge, Safari)

Étapes d'installation :

- Télécharger le code source** Cloner le dépôt GitHub ou télécharger l'archive ZIP :

```
git clone https://github.com/yasmineboulil/kyotobot-projet.git
cd kyotobot-projet
```

- Installer les dépendances Python**


```
pip install -r requirements.txt
```

Cela installera Flask et mistralai.

- (c) **Configurer la clé API (si nécessaire)** Si vous utilisez votre propre clé API Mistral, ouvrez `config.py` et remplacez la valeur de `MISTRAL_API_KEY`.

- (c) Lancer l'application

```
python app.py
```

Le serveur Flask démarrera sur `http://localhost:5000`

- (d) Ouvrir dans le navigateur Accédez à `http://localhost:5000` dans votre navigateur.

C.2 Utilisation du chatbot

Démarrage d'une conversation :

- (a) Depuis la page d'accueil, cliquez sur " Commencer l'aventure"
- (b) Le chatbot vous accueille avec un message de bienvenue
- (c) Tapez votre demande dans le champ de saisie en bas de page

Formulation des demandes : Le chatbot comprend le langage naturel. Exemples de questions possibles :

- "J'ai 60€ et une journée, je veux voir des temples"
- "Où manger des ramens pas chers ?"
- "Recommande-moi un hôtel dans le quartier de Gion"
- "Que faire à Kyoto au printemps ?"
- "Crée-moi un itinéraire romantique pour 2 jours"

Boutons de suggestion rapide : Pour faciliter la prise en main, trois boutons de suggestion sont disponibles :

- " Temples" : Lance une question sur les temples classiques
- " Inspire-moi" : Demande au bot de proposer une visite originale
- " 1 jour" : Demande un itinéraire d'une journée
- " Nouveau" : Réinitialise la conversation

Réinitialisation : Cliquez sur le bouton " Nouveau" pour effacer l'historique de conversation et recommencer.

C.3 Navigation dans le guide des lieux

Accès au guide :

- Depuis l'accueil : cliquer sur " Explorer les lieux"
- Via la navbar : cliquer sur " Lieux"

Utilisation des filtres :

- (a) En haut de la page, une barre de filtres présente les 6 catégories
- (b) Cliquer sur une catégorie (ex : "Temples ") pour n'afficher que les lieux de cette catégorie
- (c) Cliquer sur "Tous " pour réafficher tous les lieux
- (d) Les filtres modifient l'URL (`?categorie=temple`) permettant de partager un lien filtré

Consultation des détails d'un lieu :

- (a) Cliquer sur le bouton " Détails" d'une carte de lieu

- (b) La page de détail affiche : description complète, informations pratiques, conseil du guide
- (c) Deux actions possibles :
 - ” Voir sur Google Maps” : Ouvre Google Maps avec le lieu pré-recherché
 - ” Demander plus d’infos à KyotoBot” : Ouvre le chatbot avec question pré-remplie

C.4 Consultation des itinéraires prédéfinis

Accès aux itinéraires :

- Depuis l’accueil : cliquer sur ” Voir les itinéraires”
- Via la navbar : cliquer sur ” Itinéraires”

Parcourir les itinéraires :

- (a) La page présente 5 itinéraires thématiques sous forme de grandes cartes
- (b) Chaque carte indique : thème, durée, budget estimé, description courte
- (c) Cliquer sur ” Voir l’itinéraire détaillé” pour accéder au programme complet

Consultation du détail d’un itinéraire :

- (a) La page de détail présente trois sections :
 - Vue d’ensemble (durée, budget, présentation)
 - Programme détaillé (étapes heure par heure avec lieux, prix, notes)
 - Conseils pratiques (recommandations globales)
- (b) Bouton ” Personnaliser cet itinéraire avec KyotoBot” pour adapter l’itinéraire via le chatbot

C.5 Navigation générale

Navbar fixe : Une barre de navigation fixe en haut de toutes les pages permet d’accéder rapidement à :

- Accueil
- Chat ()
- Lieux ()
- Itinéraires ()

Boutons de retour : Des boutons ”← Retour” sont présents sur les pages de détail pour revenir facilement à la page précédente.

C.6 Conseils d’utilisation

- **Soyez spécifique :** Plus vous donnez de détails (budget, durée, intérêts), plus les recommandations seront précises
- **Explorez les suggestions :** Utilisez les filtres et itinéraires prédéfinis pour découvrir des lieux auxquels vous n’auriez pas pensé
- **Combinez les fonctionnalités :** Parcourez d’abord le guide des lieux, puis demandez au chatbot de créer un itinéraire incluant vos lieux préférés
- **Testez ”Inspire-moi” :** Cette fonction propose des visites originales hors des sentiers battus